Experiment 13:- Write a Program to Implement KNN Algorithm of Machine Learning on any Dataset

In [1]:
```
import pandas as pd
from sklearn.datasets import load_digits
iris = load_digits()
```

In [2]:
```
df = pd.DataFrame(iris.data,columns=iris.feature_names)
df.head()
```

Out[2]:

	pixel_0_0	pixel_0_1	pixel_0_2	pixel_0_3	pixel_0_4	pixel_0_5	pixel_0_6	pixel_0_7	pi
0	0.0	0.0	5.0	13.0	9.0	1.0	0.0	0.0	
1	0.0	0.0	0.0	12.0	13.0	5.0	0.0	0.0	
2	0.0	0.0	0.0	4.0	15.0	12.0	0.0	0.0	
3	0.0	0.0	7.0	15.0	13.0	1.0	0.0	0.0	
4	0.0	0.0	0.0	1.0	11.0	0.0	0.0	0.0	

5 rows × 64 columns

In [3]:
```
df['target'] = iris.target
df.head()
```

Out[3]:

	pixel_0_0	pixel_0_1	pixel_0_2	pixel_0_3	pixel_0_4	pixel_0_5	pixel_0_6	pixel_0_7	pi
0	0.0	0.0	5.0	13.0	9.0	1.0	0.0	0.0	
1	0.0	0.0	0.0	12.0	13.0	5.0	0.0	0.0	
2	0.0	0.0	0.0	4.0	15.0	12.0	0.0	0.0	
3	0.0	0.0	7.0	15.0	13.0	1.0	0.0	0.0	
4	0.0	0.0	0.0	1.0	11.0	0.0	0.0	0.0	

5 rows × 65 columns

In [4]:
```
iris.target_names
```

Out[4]:
```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

In [5]:
```
df[df.target==0].head()
```

Out[5]:

	pixel_0_0	pixel_0_1	pixel_0_2	pixel_0_3	pixel_0_4	pixel_0_5	pixel_0_6	pixel_0_7
0	0.0	0.0	5.0	13.0	9.0	1.0	0.0	0.0
10	0.0	0.0	1.0	9.0	15.0	11.0	0.0	0.0
20	0.0	0.0	3.0	13.0	11.0	7.0	0.0	0.0
30	0.0	0.0	10.0	14.0	11.0	3.0	0.0	0.0
36	0.0	0.0	6.0	14.0	10.0	2.0	0.0	0.0

5 rows × 65 columns

In [6]:
```
df[df.target==1].head()
```

Out[6]:

	pixel_0_0	pixel_0_1	pixel_0_2	pixel_0_3	pixel_0_4	pixel_0_5	pixel_0_6	pixel_0_7
1	0.0	0.0	0.0	12.0	13.0	5.0	0.0	0.0
11	0.0	0.0	0.0	0.0	14.0	13.0	1.0	0.0
21	0.0	0.0	0.0	2.0	16.0	16.0	2.0	0.0
42	0.0	0.0	0.0	0.0	12.0	5.0	0.0	0.0
47	0.0	0.0	0.0	1.0	11.0	9.0	0.0	0.0

5 rows × 65 columns

In [7]:
```
df[df.target==2].tail()
```

Out[7]:

	pixel_0_0	pixel_0_1	pixel_0_2	pixel_0_3	pixel_0_4	pixel_0_5	pixel_0_6	pixel_0_7
1744	0.0	2.0	11.0	16.0	13.0	2.0	0.0	0.0
1751	0.0	0.0	11.0	16.0	10.0	0.0	0.0	0.0
1780	0.0	3.0	14.0	15.0	6.0	0.0	0.0	0.0
1782	0.0	1.0	10.0	13.0	2.0	0.0	0.0	0.0
1783	0.0	0.0	15.0	13.0	1.0	0.0	0.0	0.0

5 rows × 65 columns

In [8]:
```
from sklearn.model_selection import train_test_split
```

In [9]:
```
x=df.drop(['target'],axis='columns')
x.head()
```

Out[9]:

	pixel_0_0	pixel_0_1	pixel_0_2	pixel_0_3	pixel_0_4	pixel_0_5	pixel_0_6	pixel_0_7	pi
0	0.0	0.0	5.0	13.0	9.0	1.0	0.0	0.0	
1	0.0	0.0	0.0	12.0	13.0	5.0	0.0	0.0	
2	0.0	0.0	0.0	4.0	15.0	12.0	0.0	0.0	
3	0.0	0.0	7.0	15.0	13.0	1.0	0.0	0.0	
4	0.0	0.0	0.0	1.0	11.0	0.0	0.0	0.0	

5 rows × 64 columns

In [10]:
```
y = df.target
```

In [11]:
```
x_train,x_test,y_train,y_test = train_test_split(x,y,test_size=0.2,shuffle=True)
```

In [12]:
```
from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors = 10)
knn.fit(x_train,y_train)
```

Out[12]:

▾ KNeighborsClassifier

▸ Parameters

▸ Fitted attributes

In [13]:
```
knn.score(x_test,y_test)
```

Out[13]:
```
0.9861111111111112
```

In [14]:
```
from sklearn.metrics import confusion_matrix
y_pred = knn.predict(x_test)
cm = confusion_matrix(y_test,y_pred)
cm
```

Out[14]:
```
array([[40,  0,  0,  0,  0,  0,  0,  0,  0,  0],
       [ 0, 38,  0,  0,  0,  0,  0,  0,  0,  0],
       [ 0,  0, 38,  0,  0,  0,  0,  0,  0,  0],
       [ 0,  0,  0, 35,  0,  0,  0,  0,  0,  0],
       [ 0,  0,  0,  0, 31,  0,  0,  0,  0,  0],
       [ 0,  0,  0,  0,  0, 36,  0,  0,  0,  0],
       [ 0,  0,  0,  0,  0,  0, 30,  0,  0,  0],
       [ 0,  0,  0,  0,  0,  0,  0, 38,  0,  0],
       [ 0,  2,  1,  0,  0,  0,  0,  0, 29,  0],
       [ 0,  0,  0,  2,  0,  0,  0,  0,  0, 40]])
```

In [15]:
```
from sklearn.metrics import classification_report
print( classification_report(y_test,y_pred))
```

```
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        40
           1       0.95      1.00      0.97        38
           2       0.97      1.00      0.99        38
           3       0.95      1.00      0.97        35
           4       1.00      1.00      1.00        31
           5       1.00      1.00      1.00        36
           6       1.00      1.00      1.00        30
           7       1.00      1.00      1.00        38
           8       1.00      0.91      0.95        32
           9       1.00      0.95      0.98        42

    accuracy                           0.99       360
   macro avg       0.99      0.99      0.99       360
weighted avg       0.99      0.99      0.99       360
```

In [16]:
```
df.shape
```

Out[16]: (1797, 65)

In [17]:
```
x_test.shape
```

Out[17]: (360, 64)